Algorithmique I

Formulaire et méthode

Chapitres 1 à 7 (2026-2027)

Formules

Les applications et les conventions LDA ci-dessous s'appuient sur le syllabus actuel [1] et sur les exercices de structures et de tableaux de l'édition précédente [2].

Division entière et chiffres

- **Quotient et reste :** $n = bq + r$, $0 \le r < b$. Pour $n \ge 0$ et $b > 0$, q est le quotient (`n DIV b`) et r le reste (`n MOD b`).
- **Chiffre de rang k :** $\big(n \operatorname{DIV} 10^k\big) \operatorname{MOD} 10$, où $k = 0$ désigne les unités. Pour cinq chiffres, `n DIV 10000` donne le premier, `n DIV 1000` les deux premiers et `n MOD 100` les deux derniers.
- **Recomposition :** $n = 100c + 10d + u$, où c, d, u sont les centaines, les dizaines et les unités. Le miroir est $100u + 10d + c$.

Durées et moyennes

- **Durée en secondes :** $s = 3600h + 60m + t$, où h désigne les heures, m les minutes et t les secondes. Un jour compte 86 400 s. Pour décomposer s, prendre `s DIV 3600`, `(s MOD 3600) DIV 60`, puis `s MOD 60`.
- **Vitesse :** $v = 60d/m$ en km/h, où d est la distance en km et $m > 0$ la durée en minutes. Exemple : 150 km en 90 min donnent 100 km/h.
- **Moyenne :** $\bar{x} = S/n$, où S est la somme et $n > 0$ le nombre de valeurs retenues. Pour l'exercice du bilan, la note est $0{,}25B + 0{,}75E$ si $B > E$, et E sinon. B désigne le bilan et E l'examen.
- **Prix net :** $P = q\,p(1 - r)$, avec q la quantité, p le prix unitaire et r le taux de remise (5 % donne $r = 0{,}05$).

Géométrie et équations

- **Aires :** $bh/2$ pour un triangle de base b et de hauteur h ; $d^2/2$ pour un carré de diagonale d, dont le côté vaut $d/\sqrt{2}$. Exprimer les longueurs dans la même unité.
- **Distance au carré :** $d^2 = (x_2 - x_1)^2 + (y_2 - y_1)^2$, pour deux points de coordonnées (x_1, y_1) et (x_2, y_2). Deux disques de rayons r_1, r_2 se rencontrent si $d^2 \le (r_1 + r_2)^2$, où d est la distance entre leurs centres. Pour les circonférences seules, exiger aussi $d^2 \ge (r_1 - r_2)^2$.
- **Second degré :** pour $ax^2 + bx + c = 0$, a, b, c sont les coefficients, x l'inconnue et $\Delta = b^2 - 4ac$. Si $a \neq 0$, $\Delta < 0$ donne zéro racine réelle, $\Delta = 0$ donne $-b/(2a)$, et $\Delta > 0$ donne $(-b \pm \sqrt{\Delta})/(2a)$. Traiter le cas $a = 0$ séparément.

Calculs sur les entiers

- **Factorielle :** $0! = 1$ et $n! = n(n-1)!$ pour un entier $n \ge 1$.
- **PGCD et PPCM :** remplacer (a, b) par $(b, a \bmod b)$ tant que $b \neq 0$, pour deux entiers non négatifs. Le dernier a est le PGCD. Pour $a, b > 0$, $\operatorname{PPCM}(a, b) = \big(a / \operatorname{PGCD}(a, b)\big)b$. Exemple : le PGCD de 48 et 18 vaut 6 et leur PPCM vaut 144.

Méthodes

1. Lire et écrire le LDA

- Déclarer et initialiser avant d'utiliser. Choisir `int` pour un entier, `float` pour une mesure, `Boolean` pour un test et `String` pour du texte. L'indentation délimite les blocs.
- Utiliser `=` pour affecter, `==` pour comparer et `/` pour une division réelle. `read` lit dans l'ordre demandé et `print` affiche.
- Pour un module, préciser le nom, le type et l'ordre des paramètres, les préconditions et le résultat. Garder un seul `return` final ; si plusieurs branches déterminent le résultat, passer par une variable locale. Un module `void` ne renvoie pas de valeur.

```
float aireCarre(float cote)
    float aire
    aire = cote*cote
    return aire
```

2. Tracer une exécution

Faire une colonne par variable et une pour les sorties. Noter « non définie » à la déclaration et ne mettre à jour que les variables modifiées. Tracer les appels séparément : `calculs(a,2*b)` reçoit $1, 8$ si $a = 1, b = 4$. Seul le résultat du `return` rejoint l'expression appelante. Noter les affichages dans leur ordre réel.

3. Extraire, remplacer et parcourir des chiffres

Isoler les chiffres avant de les comparer ou de les remplacer. Un nombre ne conserve pas ses zéros en tête : le miroir de 120 vaut 21, alors que la chaîne attendue peut être « 021 ». Pour construire le miroir pendant un parcours, ajouter chaque chiffre à droite par `miroir = 10*miroir + chiffre`. Un reste nul modulo 2 indique la parité. Pour un entier négatif, traiter le signe avant d'extraire les chiffres.

```
int sommeChiffres(int n)
    int somme
    somme = 0
    while n > 0
        somme = somme + n MOD 10
        n = n DIV 10
    return somme
```

Cette version attend $n \geq 0$. Tester 902 (résultat 11) et 0 (résultat 0).

4. Choisir des conditions et tester leurs frontières

- `AND` exige les deux conditions, `OR` au moins une et `NOT` inverse la valeur. Par exemple, $\neg(a > b)$ équivaut à $a \leq b$. Placer la garde avant le calcul dans `b != 0 AND a MOD b == 0`.
- Une chaîne `else if` s'arrête à la première condition vraie. Dans un `switch`, écrire les valeurs suivies de deux-points et prévoir `default` si nécessaire.
- Écrire les intervalles puis tester chaque égalité. Pour une ouverture de 9 h à midi puis de 13 h à 18 h, utiliser $[9, 12[\cup[13, 18[$ si les heures de fin sont exclues. Distinguer la remise sur toute une commande d'un tarif progressif par tranches.

```
if quantite < 5
    taux = 0
else if quantite < 10
    taux = 0.05
else if quantite < 50
    taux = 0.10
else
    taux = 0.15
prixNet = quantite*prixUnitaire*(1 - taux)
```

5. Comparer et permuter

Initialiser un extremum avec une valeur effectivement lue, pas avec zéro par défaut. Pour garder le premier indice à égalité, n'accepter qu'une amélioration stricte ; pour garder le dernier, accepter aussi l'égalité. Pour permuter, sauvegarder une valeur avant de l'écraser. Pour tester un triangle défini par deux angles, calculer le troisième, vérifier que les trois sont positifs, puis comparer les paires.

6. Manipuler des dates et des heures

- Valider l'année et le mois avant de valider le jour. Une année bissextile est multiple de 400, ou multiple de 4 sans être multiple de 100. Février compte 28 ou 29 jours ; avril, juin, septembre et novembre en comptent 30 ; les autres 31. Pour le jour de semaine, fixer une date de référence avant d'utiliser le modulo 7.
- Comparer des dates dans l'ordre année, mois, jour. Propager les retenues pour faire avancer une heure. Une heure de la journée revient à zéro à minuit, une durée peut dépasser 24 h.
- Pour un écart, soustraire les heures converties en secondes. Ajouter 86 400 si l'écart est négatif et que la fin est supposée le lendemain. Avec cette convention, deux heures identiques donnent un écart nul.
- En AM/PM, appliquer le modulo 12, afficher 12 à la place de 0, garder deux chiffres pour les minutes, et passer de AM à PM à midi.

7. Choisir une boucle et cumuler

Utiliser `for` si les bornes sont connues et `while` si l'arrêt dépend des données. Avec `for i from a to b by pas`, la borne finale est incluse si elle est atteinte. Vérifier le sens du pas et la progression. Pour un affichage imbriqué, séparer les lignes, les colonnes et le saut de ligne. Une somme démarre à 0, un produit à 1 et un compteur n'avance que si le critère est vrai. Exclure la sentinelle du calcul. Pour tester si un entier n est premier, écarter $n < 2$, puis tester les diviseurs d tant que $d^2 \leq n$.

8. Regrouper les données d'une même entité

Définir les champs d'une structure puis préciser ceux qu'on lit ou qu'on modifie : `date.jour`, `point.x`, `cercle.centre.x`. Initialiser chaque champ lu. Utiliser une copie si la valeur d'origine doit rester intacte. Les exemples de notation pointée reprennent [2].

9. Parcourir, rechercher et modifier un tableau

- Distinguer la capacité du nombre n d'éléments utilisés. Avec des indices à partir de zéro, vérifier $0 \leq i < n$ avant tout accès à `t[i]`. Une moyenne ou un extremum exige un tableau non vide. Pour calculer entre deux extrema, remettre leurs indices dans l'ordre et inclure les deux bornes.
- Pour trouver une valeur dans un tableau trié par dichotomie, réduire l'intervalle de candidats à chaque étape. Pour la première occurrence, mémoriser l'égalité et continuer à gauche. Utiliser la valeur d'échec prévue par la consigne.

- Pour tester l'ordre, comparer les voisins. Pour inverser, échanger `t[i]` et `t[n-1-i]` dans la première moitié. Pour décaler à droite, recopier en partant de la droite.
- Initialiser les compteurs d'occurrences à zéro. En présence de doublons, traiter les correspondances exactes d'abord puis apparier chaque occurrence restante au plus une fois. Pour compter `OXO` sans recouvrement, avancer de trois en cas de succès et d'un sinon. Pour garder toutes les positions du minimum, vider la liste à chaque nouveau minimum strict et ajouter les égalités.

10. Vérifier avant de rendre

Relire le contrat et le format de sortie. Tester une entrée ordinaire, une frontière et le plus petit cas permis. Pour une boucle, vérifier zéro, un et plusieurs passages. Pour une trace, comparer les étapes et les affichages, puis le résultat.

Références

[1] ESI, HE2B, *Algorithmique I : syllabus 2026-2027*, Bruxelles, Belgique, 2026, ch. 1 à 5.

[2] ESI, HE2B, *Algorithmique : syllabus 2025-2026*, Bruxelles, Belgique, 2025, ch. 5 et 7.